Science in Information Technology Letters
Vol. 6., No. 2, November 2025, pp. 66-88
ISSN 2722-4139
http://pubs2.ascee.org/index.php/sitech

SHAP-based interpretable deep learning framework for phishing website detection

Rizki Wahyudi [a,1,*], Altayeb Alhadi Dawai [b,2], Deris Stiawan [c,3], Andri Pranolo [d,4], Yingchi Mao [e,5], Amam Hossain Bagdade [f,6]

[a] Universitas Amikom Purwokerto, Purwokerto, Indonesia
[b] University of Algarve, Faro, Portugal
[c] Computer System, Universitas Sriwijaya, Palembang, Indonesia
[d] Informatics Department, Universitas Ahmad Dahlan, Yogyakarta, Indonesia
[e] College of Computer and Information, Hohai University, Nanjing, China
[f] EEE Department, Presidency University, Dhaka, Bangladesh
[1] rizki.key@gmail.com; [2] altayebalhadi53@gmail.com; [3] deris@unsri.ac.id; [4] andri.pranolo@tif.uad.ac.id;
[5] yingchimao@hhu.edu.cn; [6] amambagdadee@gmail.com
* Corresponding Author

ARTICLE INFO

Article history
Received September 20, 2025
Revised November 09, 2025
Accepted November 12, 2025

Keywords
Phishing detection
Deep Learning
SHAP
Interpretability
Website Classification

ABSTRACT

Phishing attacks are increasingly complex and challenging to detect, posing a substantial challenge for cybersecurity professionals. This paper proposes a SHAP-based Interpretable Deep Learning Framework aimed at identifying phishing websites, emphasizing transparency in the models' decision-making processes. The five deep learning architectures evaluated were the Artificial Neural Network (ANN), Multi-Layer Perceptron (MLP), Autoencoder, Convolutional Neural Network (CNN), and Bidirectional Long Short-Term Memory (Bi-LSTM). The evaluation process employed several metrics, including accuracy, precision, recall, F1-score, area under the receiver operating characteristic curve (AUC-ROC), and model interpretability through SHAP (Shapley Additive Explanations). The experimental results demonstrate that the Bi-LSTM algorithm achieves superior performance, with an accuracy of 97.03%, a recall of 98.44%, and an F1-score of 97.36%. SHAP visualization successfully identifies the most significant aspects in phishing classification, thereby enhancing both transparency and trust in the model. This method not only efficiently detects phishing but also provides clear explanations for individuals lacking technical knowledge. This study contributes to the development of a deep learning-based phishing detection system that is both accurate and computationally understandable.

© 2025 The Author(s).
This is an open access article under the CC–BY-SA license.

1. Introduction

Phishing attacks have emerged as one of the most widespread and harmful cyber hazards in today's ever-changing digital age. Phishing is a type of fraud in which scammers impersonate reputable websites to obtain sensitive user information such as login credentials, credit card information, and other personal information. According to the Anti-Phishing Working Group (APWG), phishing operations have lately

reached historic levels, with more than one million attacks documented in a single quarter of 2022 [1]. As the complexity and volume of phishing attacks increase, the need for accurate and reliable automated detection systems becomes increasingly urgent.

Identifying phishing websites uses various techniques, such as list-based, visual similarity, heuristic, machine learning [2], [3], data mining [4], and deep learning, [5]. Various machine learning-based approaches have been developed to detect phishing sites, including Decision Tree, Random Forest, and Support Vector Machine. However, many of these models depend significantly on manually engineered features, which may not generalise effectively to new attack patterns or emerging phishing techniques [6]–[9].

Recently, deep learning has demonstrated remarkable capabilities in capturing non-linear patterns and automatically representing features from raw data [10], [11]. In phishing detection tasks, deep learning has shown improved accuracy and reduced dependency on manual feature extraction, making it suitable for identifying sophisticated or obfuscated phishing URLs [12]. Models such as Artificial Neural Network (ANN), Multi-Layer Perceptron (MLP), Autoencoder, Convolutional Neural Network (CNN), and Bidirectional Long Short-Term Memory (Bi-LSTM) have been successfully used in various fields, like detecting unusual patterns and enhancing cybersecurity [13]–[16]. However, despite having high performance, deep learning models are often criticised as black boxes due to their lack of interpretability [17], [18]. This lack of transparency poses a significant barrier in cybersecurity applications where explainability is crucial for analyst trust, auditing, and compliance [19].

To address this challenge, recent research has begun to adopt model interpretability techniques to provide a more profound understanding of the predictions generated by deep learning models. Some common strategies include SHapley Additive Explanations (SHAP), Local Interpretable Model-Agnostic Explanations (LIME), and Grad-CAM. SHAP elucidates the contribution of each input feature to the model's choice, grounded in the Shapley value theory from cooperative game theory [20]–[22]. In the context of web phishing detection, SHAP allows for the identification of important features, such as URL length, the presence of special symbols, or domain structure that significantly contribute to the classification as a phishing site. Interpretability techniques not only enhance user trust in automated detection systems but also provide valuable insights in designing more effective mitigation strategies against phishing attacks [23]–[25].

Few studies have integrated model interpretation techniques such as SHAP into a comparative framework that evaluates multiple deep architectures, despite the expanding corpus of work on deep learning for phishing detection [7], [26], [27]. This study aims to provide an interpretable deep learning framework for phishing website detection by integrating SHAP explanations into the evaluation of several architectures. This work departs from previous research that primarily focuses on classification accuracy, instead emphasizing prediction performance and model transparency through SHAP-based analysis. Five deep learning architectures ANN, MLP, Autoencoder, CNN, and Bi-LSTM are rigorously evaluated based on classification metrics and explainability, offering insights into the influence of input data on model predictions. This dual focus on performance and interpretability addresses a critical deficiency in current literature and improves the reliability and usability of phishing detection systems.

2. Method

This research aims to evaluate and compare the performance of five deep learning models in detecting phishing websites. The five models are Artificial Neural Network (ANN), Multi-Layer Perceptron (MLP), Autoencoder Neural Network, Convolutional Neural Network (CNN), and Bidirectional Long Short-Term Memory (Bi-LSTM). The research was conducted using a quantitative experimental approach, utilizing a processed dataset divided into training and testing data. Fig. 1 illustrates the stages of the research.

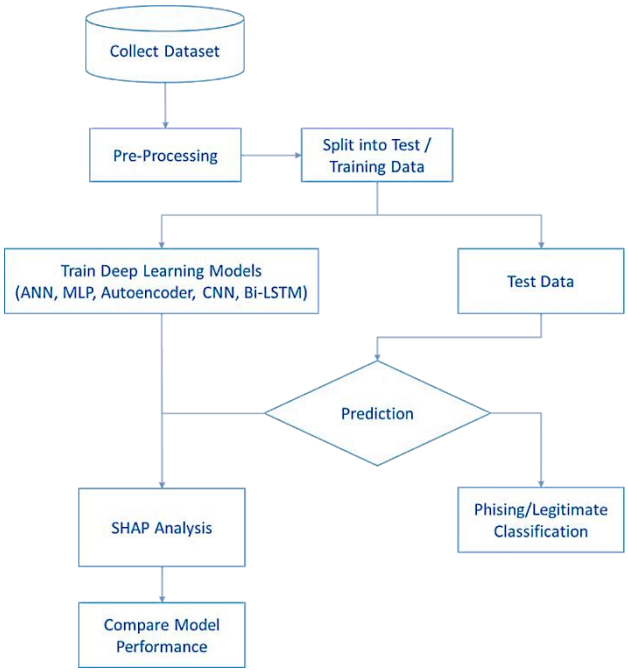


Fig. 1. Research Stages

2.1. Dataset

The dataset that was utilized was obtained from the platform within the UCI Machine Learning Repository [28]. This dataset includes target labels in the form of binary values (0 stands for not phishing and 1 stands for phishing), as well as numeric features that indicate the qualities of URLs and web content. The data were normalized after the index column was removed to make the scales of the features more comparable to one another. The dataset was then stratified to ensure that the proportion of each class was maintained in a balanced manner, and it was separated into two parts: 75% for training data and 25% for validation data.

2.2. Data Pre-Processing

Normalization: Input features are normalized using z-score normalization, which is done by subtracting the mean and dividing it by the standard deviation [29], [30].

Data Reshape: The Bi-LSTM model transforms the data into a three-dimensional format (samples, timesteps, features), with timesteps set at 1, as the data is not sequential in time [31].

2.3. Model Architecture

To ensure that the decision-making process is open and transparent, each model has been explicitly created and tuned to investigate its potential for categorization against phishing data. Additionally,

interpretability approaches have been incorporated into the structure of the models. Fig. 2 illustrates both the design and the interpretability technique that was implemented.

- Artificial Neural Network (ANN) : The ANN model consists of several dense layers with the ReLU activation function, accompanied by dropout and batch normalization to reduce overfitting [32], [33].
- Multi-Layer Perceptron (MLP) : The MLP architecture is almost similar to ANN but has the number of neurons and hidden layers adjusted exploratively [34], [35].
- Autoencoder Neural Network : An autoencoder is built with two main parts: (i) Encoder: Two LSTM layers that filter out important features and construct a low-dimensional latent representation. (ii) Decoder: A symmetric LSTM structure to reconstruct the input. The latent representation from the encoder is used as input to an additional classification model [36].
- Convolutional Neural Network (CNN) : A CNN model is composed of one or more 1D convolutional layers that use 'same' padding, followed by a pooling layer and a dense layer for classification [37].
- Bidirectional LSTM (Bi-LSTM) : The Bi-LSTM model uses two-way sequential processing to capture feature relationships from two directions. This model uses two Bi-LSTM layers followed by a dense layer [38], [39].

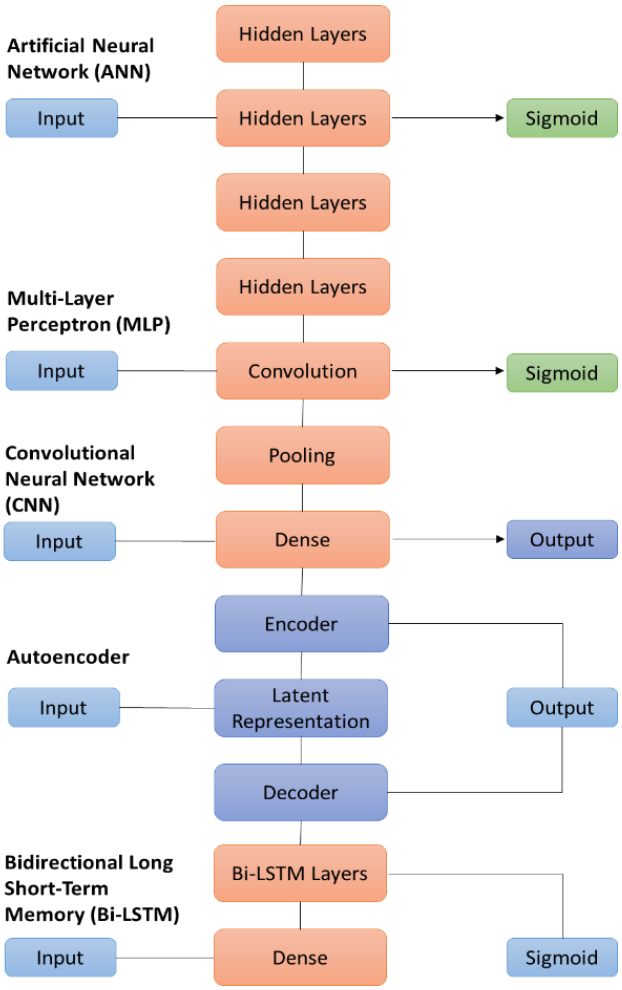


Fig. 2. Architecture and interpretability strategies

2.4. The model training process

Using the Adam optimizer, each model was trained with either the binary_crossentropy or mean_squared_error loss function (for autoencoders). To avoid overfitting, the training phase was limited to 100 epochs and ended early [40]. Batch size is set at 128.

2.5. Model Evaluation

At this stage, each algorithm is trained using the training data. Evaluation of the Method: Performance measurements encompassing accuracy, precision, recall, F1-score, Area Under the Precision-Recall Curve (AUPRC), and Area Under the Receiver Operating Characteristic Curve (AUC-ROC) [41]–[44], The amounts of time of training and testing are employed to evaluate the model on the test data.

- Accuracy measures how often the classifier is actually correct in predicting positive and negative classes.

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \quad (1)$$

- Precision measures how often the positive class predictions are correct.

$$Precision = \frac{TP}{TP+FP} \quad (2)$$

- Recall measures how often the positive class is correctly predicted by the model.

$$Recall = \frac{TP}{TP+FN} \quad (3)$$

- The F1-score is the balanced average of precision and recall, offering a comprehensive assessment of the methods performance.

$$F_1 - score = 2\,x\,\frac{Precision\ x\ Recall}{Precision+Recall} \quad (4)$$

- The area beneath the ROC curve is quantified by AUC-ROC, which depicts the correlation between the true positive rate and the FPR, indicating the models ability to differentiate between positive and negative classifications.

2.6. SHAP-Based Interpretability

We utilize SHAP (SHapley Additive exPlanations) as an additive feature-attribution framework to measure the contribution of each feature to a model's prediction. SHAP consolidates various previous attribution techniques into a singular game-theoretic framework and is distinguished by advantageous attributes, including local accuracy, handling of missing data, and consistency, which distinctly define the attributions under the category of additive methods [45].

For an instance x with feature set F ($|F| = M$), the Shapley value $\phi j(x)$ for feature j

$$\phi j(x) = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(M-|S|-1)!}{M!} (fx(S \cup \{j\}) - fx(S)) \quad (5)$$

The model output is denoted by the notation $f\ x\ (\ S\ )$ f x (S) when just the features that are present in coalition S S are present, while other features are missing. According to the Shapley axioms, this

ensures that the prediction is distributed among the characteristics in a manner that is theoretically objective [46].

The global importance of a feature j is expressed as the mean absolute Shapley value across all samples:

$$\phi_j = \frac{1}{n} \sum_{i=1}^{n} |\emptyset_j (xi)| \quad (6)$$

It enables a global structure that is formed from a large number of local explanations and aggregates local attributions into a stable global ranking [46].

DeepSHAP is employed for differentiable deep networks: ANN, MLP, CNN, and Bi-LSTM, while KernelSHAP is applied when layers or gradients are unsupported. Explanations are computed exclusively on the held-out test set to prevent leakage. The background distribution consists of a stratified sample drawn from the standardized training data. Unless noted otherwise, the explained quantity is the logit of the positive class, phishing, to mitigate probability saturation. SHAP analyses are presented only for the best-performing model determined during model selection, with AUPRC as the primary criterion and Macro-F1 used to break ties. Global summaries report mean absolute SHAP values, and distributional views, such as beeswarm plots, are provided in the Results section. SHAP explanations are computed and presented for the best-performing model selected in the model-selection step, AUPRC primary, Macro-F1 tie-breaker.

SHAP analyses were conducted exclusively on the best-performing model (Bi-LSTM) using the AUPRC metric as the primary selection criterion and Macro-F1 as the tie-breaker. Global summaries are reported using mean absolute SHAP values, while local explanations are visualized via beeswarm and force plots.

2.7. Reproducibility and Configuration Checklist

To ensure experimental transparency and reproducibility, all configurations were standardized across models. The following checklist summarizes key parameters and computational settings used in this study, following reproducibility principles for deep-learning experiments show in Table 1. [46], [47].

Table 1. Reproducibility and Configuration Checklist

Aspect	Configuration
Dataset Source	UCI Phishing Websites Dataset (11.055 records)
Target / Label	Target (binary: 0=legitimate, 1=phishing)
Data Split	Stratified train/valid = **75% / 25%** (random_state=42)
Index Handling	Drop column index (tolerant: errors='ignore')
Feature Scaling	**Z-score normalization**: X=(X−μ)/σ
Reshape (for LSTM)	(samples,1,features)
Deep-Learning Framework	TensorFlow **2.12**, Keras API
Programming Environment	**Google Colab Pro+** (Python **3.10**, NumPy **1.25**, Pandas **2.1**)
Hardware (Runtime Type)	NVIDIA **A100** (40 GB)
Runtime Memory	High-RAM instance (≈ **25 GB**)
Random Seed	**42** (data split & init)
Optimizer	**Adam** (lr=0.001, β_1=0.9, β_2=0.999)
Epoch Limit	**100** (with EarlyStopping, patience=**10**, restore_best_weights=True)
Batch Size	**128** (Bi-LSTM, CNN, ANN/MLP); 512 (Autoencoder)
Loss Function	**Binary Cross-Entropy** (klasifikasi); **MSE** (rekonstruksi Autoencoder)
Primary Metric (Model Selection)	**AUPRC (Average Precision)**
Additional Metrics	Accuracy, Precision, Recall, **F1, ROC-AUC**

Aspect	Configuration
Explainability Method	**SHAP - GradientExplainer** (for differentiable networks); **KernelSHAP** as fallback
Explained Quantity	**Logit** of the positive (phishing) class to avoid probability saturation
Background Sample (SHAP)	≈ **500 stratified** samples from **training data** (no leakage)
Visualization Tools	Matplotlib, seaborn (plots); SHAP **0.42.1**
Reproducibility Note	All experiments executed on Colab Pro+ runtime with fixed software versions and seeds to ensure result replication

All model scripts, SHAP visualization routines, and configuration logs were executed in identical environments to avoid hardware or library-induced variations. This setup ensures that the comparative performance and interpretability outcomes can be reliably reproduced on equivalent computational systems.

3. Results and Discussion

3.1. Training Architecture and parameters

3.1.1. Artificial Neural Network (ANN) Model Architecture

This study employed a sequentially constructed Artificial Neural Network (ANN) model featuring multiple layers organized hierarchically as show in Table 2. The input layer accommodates nnn features, reflecting the quantity of features subsequent to the preprocessing of the phishing dataset. The ANN architecture comprises two hidden layers, each containing 512 neurons and employing a ReLU activation function. Each concealed layer is succeeded by batch normalization and dropout at a rate of 0.3 to mitigate overfitting. The output layer comprises a single neuron utilizing a sigmoid activation function to generate a binary probability output (phishing or non-phishing). The model was constructed with the binary crossentropy loss function and the Adam optimizer. With a batch size of 512, the training process was carried out for a maximum of 200 epochs. An early stopping strategy was employed to prevent overfitting and expedite training, with a patience value of 20 and a minimum delta of 0.01. Model evaluation was conducted utilizing accuracy, precision, recall, F1-score, and ROC-AUC metrics.

Table 2. ANN Model Architecture

No	Layer	Neuron	Activation
1	Input + BatchNorm	-	-
2	Dense	512	ReLU
3	BatchNormalization	-	-
4	Dropout	-	-
5	Dense	512	ReLU
6	BatchNormalization	-	-
7	Dropout	-	-
8	Dense	1	Sigmoid

3.1.2. Multi-Layer Perceptron (MLP) Model Architecture

The Multi-Layer Perceptron (MLP) model in this study is implemented using the Keras Sequential API with a layered structure resembling the standard artificial neural network architecture as show in Table 3. This model is designed to handle binary classification between phishing and non-phishing sites. The model input consists of a number of features resulting from the dataset preprocessing, with an input

shape of n features. There are two hidden layers, each with 512 neurones, equipped with the ReLU activation function to introduce non-linearity.

Each hidden layer was followed by batch normalisation and dropout regularisation with a ratio of 0.3 to prevent overfitting. The output layer consists of a single neurone with a sigmoid activation function, which is responsible for generating binary classification probabilities. The model was compiled using the binary crossentropy loss function and optimised with the Adam Optimiser algorithm. The training process was carried out for a maximum of 200 epochs with a batch size of 512, and using early stopping with a patience of 20 and a min_delta of 0.01 to stop training when validation performance no longer significantly improves. The model was evaluated using the following metrics: accuracy, precision, recall, F1-score, and ROC-AUC.

Table 3. MLP Model Architecture

No	Layer	Neuron	Activation
1	Input + BatchNorm	-	-
2	Dense	512	ReLU
3	BatchNormalization	-	-
4	Dropout	-	-
5	Dense	512	ReLU
6	BatchNormalization	-	-
7	Dropout	-	-

3.1.3. Autoencoder Model Architecture

The autoencoder model is used in this study as a representation learning-based approach to extract latent features from input data prior to classification. The architecture consists of two main components the autoencoder and the classifier, which are trained separately as show in Table 4.

- Autoencoder Architecture

  An autoencoder is built with two parts: an encoder and a decoder. The encoder reduces the dimensionality of the original features to a lower-dimensional compressed representation (latent space), while the decoder reconstructs the original input from the latent representation. The encoder structure consists of two dense layers of 128 and 64 neurons, respectively, each using ReLU activation, followed by batch normalization. The latent representation layer consists of 32 neurons with ReLU activation. The decoder has a symmetric structure, with two dense layers of 64 and 128 neurons, also using ReLU activation and batch normalization. The output layer uses a sigmoid activation function, with as many neurons as the input features, to reconstruct the input data.

- Classifier Architecture

  The latent features generated from the encoder are then used as input to a separate classifier tasked with phishing classification. The classifier architecture consists of two hidden layers: 128 and 64 neurons, each using ReLU, followed by batch normalization and dropout with a ratio of 0.3. The output layer consists of one neuron with sigmoid activation. The autoencoder is compiled using the mean squared error (MSE) loss function and optimized using the Adam optimizer, while the classifier uses binary crossentropy as the loss function. Both models are trained with early stopping to avoid overfitting.

Table 4. Autoencoder and Classifier Model Architecture

No	Component	Layer	Neuron	Activation
1	Encoder	Dense	128	ReLU
2		BatchNormalization	-	-
3		Dense	64	ReLU
4		BatchNormalization	-	-
5		Dense (LatentSpace)	32	ReLU
6	Decoder	Dense	64	ReLU
7		BatchNormalization	-	-
8		Dense	128	ReLU
9		BatchNormalization	-	-
10		Dense	n fitur	Sigmoid
11	Classifier	Dense	128	ReLU
12		BatchNormalization	-	-
13		Dropout	-	-
14		Dense	64	ReLU
15		BatchNormalization	-	-
16		Dropout	-	-
17		Dense	1	Sigmoid

3.1.4. Convolutional Neural Network (CNN) Model Architecture

The Convolutional Neural Network (CNN) model in this study was built using a sequential approach and aimed to process tabular phishing feature data that had been reshaped into a 3-dimensional format so that it could be processed by a one-dimensional convolutional layer (Conv1D). The CNN architecture consists of two convolutional layers as show in Table 5. The first layer uses 64 filters with a kernel size of 3 and a ReLU activation function, followed by Batch Normalization and MaxPooling1D to reduce the feature dimensionality. The second layer uses 128 filters with a kernel size of 3, also followed by Batch Normalization and MaxPooling1D.

After passing through the convolutional layers, the data is flattened into a one-dimensional vector and then processed by a dense layer with 128 neurons and ReLU activation. To prevent overfitting, dropout was used with a dropout rate of 0.3. The output layer consists of a single neuron with a sigmoid activation function to generate a binary probability (phishing or non-phishing). The model was compiled using the binary crossentropy loss function and the Adam optimizer. The training process was carried out for a maximum of 100 epochs with a batch size of 512. Early stopping was used with a patience value of 10 and restore_best_weights was enabled. Model evaluation was carried out using accuracy, precision, recall, F1-score, and ROC-AUC metrics.

Table 5. CNN Model Architecture

No	Layer	Neuron / Filter	Activation
1	Input + Reshape	-	-
2	Conv1D	64 filter, kernel 3	ReLU
3	BatchNormalization	-	-
4	MaxPooling1D	pool size = 2	-

No	Layer	Neuron / Filter	Activation
5	Conv1D	128 filter, kernel 3	ReLU
6	BatchNormalization	-	-
7	MaxPooling1D	pool size = 2	-
8	Flatten	-	-
9	Dense	128	ReLU
10	Dropout	-	-
11	Dense	1	Sigmoid

3.1.5. Bidirectional Long Short-Term Memory (Bi-LSTM) Model Architecture

A Bidirectional Long Short-Term Memory (Bi-LSTM) model is built to capture temporal patterns and dependencies between features from phishing data that has been re-represented in the form of one-dimensional sequences as show in Table 6. The input data is reshaped into a 3-dimensional form with a structure (samples, timesteps, features), where timesteps = 1. The Bi-LSTM architecture consists of two main bidirectional LSTM layers. The first layer uses 64 units with the configuration return_sequences=True to flow information to the next LSTM layer. The second layer uses 32 units and returns the final output of the sequence. After the sequential processing process, the output is continued to the dense layer twice: first with 128 neurons and ReLU activation, then with 64 neurons and ReLU activation. Each of these layers is followed by batch normalization and a dropout rate of 0.3 to reduce the risk of overfitting. The output layer consists of one neuron with a sigmoid activation function to produce a binary probability (phishing or non-phishing). The model was compiled using binary crossentropy and the Adam optimizer. Training was performed for a maximum of 100 epochs with a batch size of 128 and using early stopping with a patience of 10.

Table 6. Bi-LSTM Model Architecture

No	Layer	Unit / Neuron	Activation
1	Input + Reshape	-	-
2	Bidirectional LSTM	64 (return_sequences)	ReLU
3	Bidirectional LSTM	32	ReLU
4	Dense	128	ReLU
5	BatchNormalization	-	-
6	Dropout	-	-
7	Dense	64	ReLU
8	BatchNormalization	-	-
9	Dropout	-	-
10	Dense	1	Sigmoid

3.2. Model Training Process

Fig. 3. Analysis of the loss over epochs graph shows that all proposed deep learning models experienced a consistent decrease in loss values during the training process, indicating good convergence. The ANN and MLP models showed a steady decreasing trend in training loss and validation loss, although there was a slight difference in validation loss at the end of training, indicating a small possibility of overfitting. The autoencoder model showed very stable training behavior with nearly overlapping training and validation loss curves, indicating excellent generalization. The CNN also displayed a smooth and controlled loss decrease pattern, with a small difference between training and validation loss throughout the epochs, reflecting the model's resilience to overfitting. Meanwhile, the

Bi-LSTM achieved rapid convergence in fewer epochs than the other models, with a loss pattern that decreased sharply at the beginning and stabilized in the final phase, indicating efficiency in the learning process. Overall, the loss over epochs trend in all models supports the effectiveness of the developed framework in detecting phishing sites accurately and stably.

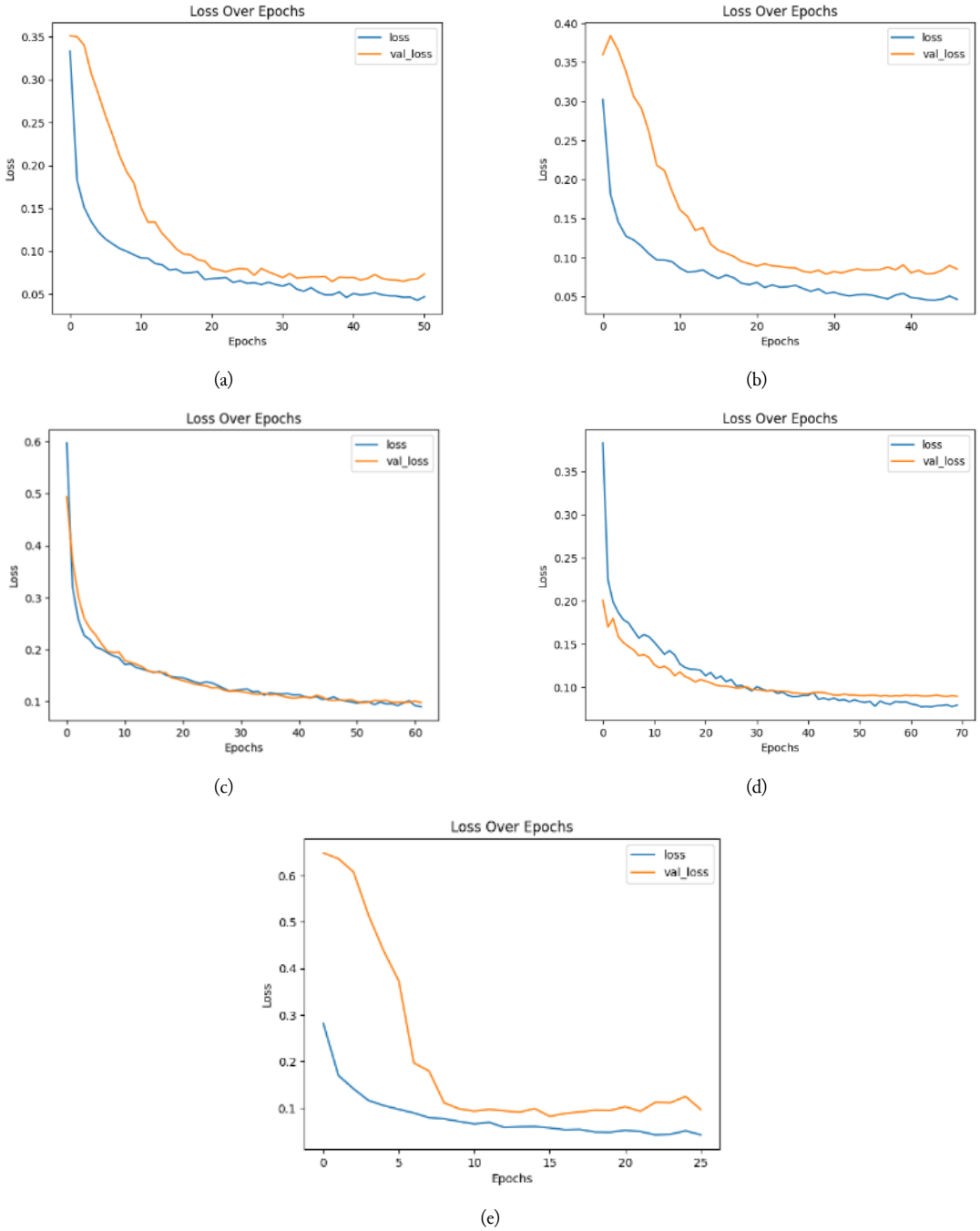


Fig. 3. Loss Over Epochs Model (a) ANN, (b) MLP, (c) Autoencoder, (d) CNN, (e) Bi-LSTM

Analysis of model accuracy during the training process shows a consistent pattern across each model type tested: Artificial Neural Network (ANN), Multi-Layer Perceptron (MLP), and Autoencoder show in Fig. 4. In the ANN model, both training accuracy (binary_accuracy) and validation accuracy (val_binary_accuracy) show a steady increase with increasing epochs, reaching a maximum near 0.98.

Although val_binary_accuracy exhibits slight fluctuations toward the end of training, both remain at a high level, reflecting the model's good generalization.

On the other hand, the MLP model performs similarly, with stable accuracy and a gradual increase until it reaches 0.96. This pattern suggests that a higher epoch count provides a slight increase in accuracy, but the difference between training and validation accuracy indicates potential overfitting that should be considered. Finally, the autoencoder model demonstrates fairly high accuracy, with a faster plateau, reaching 0.95 in less than 30 epochs. Although validation accuracy remains below training accuracy, these results indicate that the models perform well in representing the data through the learned features. Overall, all models show a positive trend in accuracy improvement, albeit with varying convergence speeds and consistency.

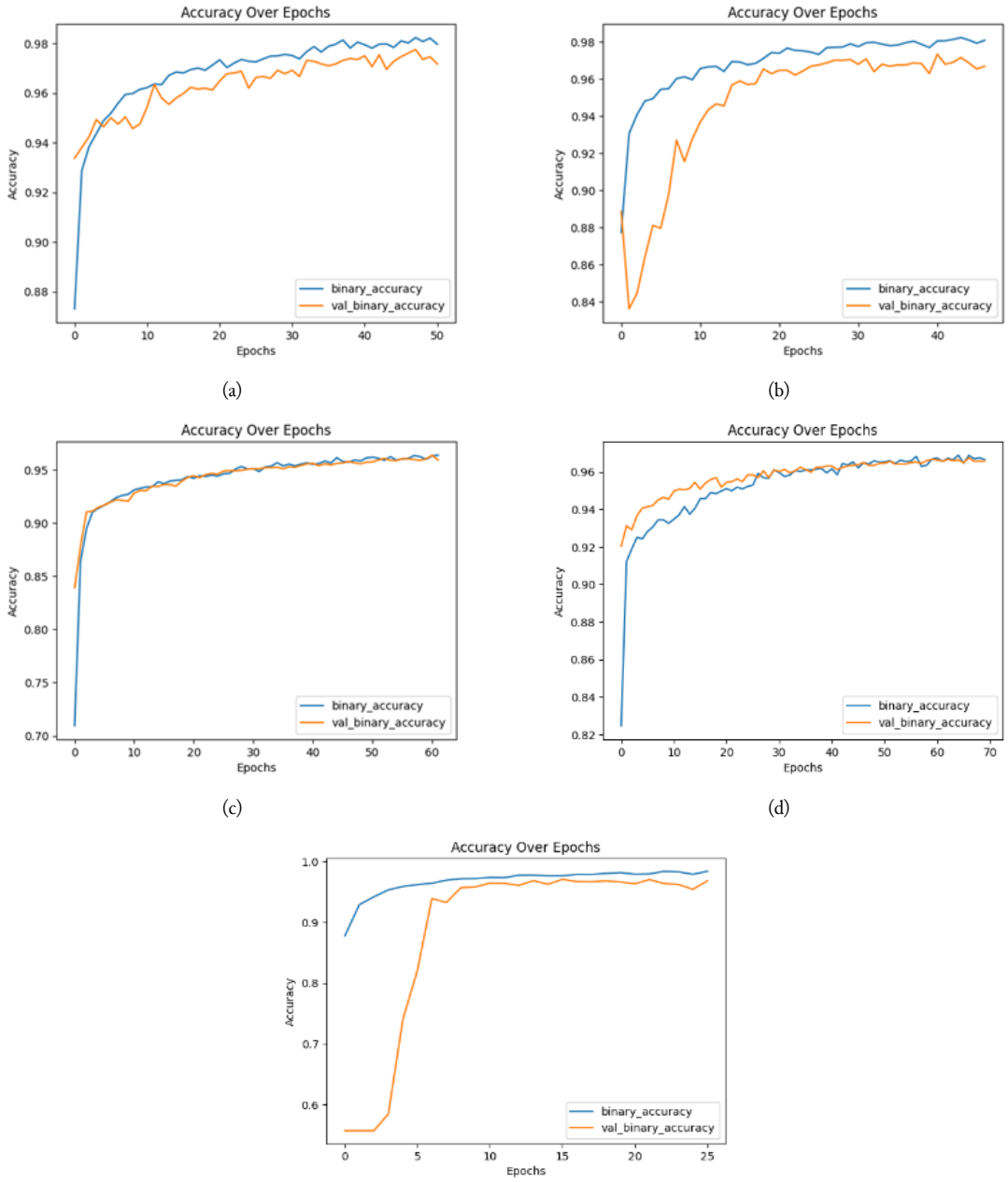


Fig. 4. Accuracy Over Epochs Model (a) ANN, (b) MLP, (c) Autoencoder, (d) CNN, (e) Bi-LSTM

3.2.1. Model Performance Evaluation

This study compares five deep learning architectures: Artificial Neural Network (ANN), Multi-Layer Perceptron (MLP), Autoencoder, Convolutional Neural Network (CNN), and Bidirectional Long Short-Term Memory (Bi-LSTM) in detecting phishing sites as show in Fig. 5. Each model was trained using a structured phishing dataset, and its performance was tested based on five key metrics: accuracy, precision, recall, F1-score, and ROC-AUC.

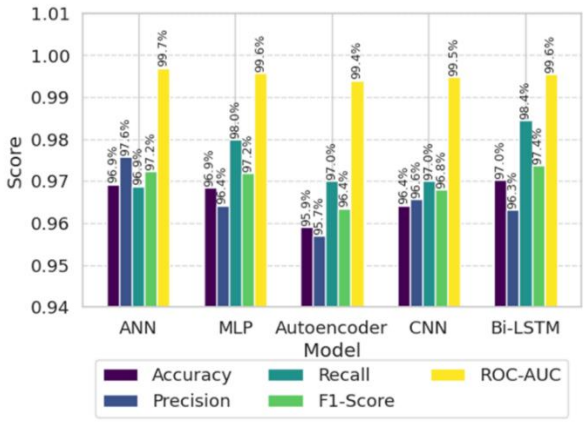


Fig. 5. Comparison of the performance of different Model

In this study, the evaluation of the five deep learning models demonstrated highly competitive performance in detecting phishing sites. The Bi-LSTM model achieved the best performance with an accuracy of 97.03%, a recall of 98.44%, and an F1-score of 97.36%, demonstrating its superior ability to accurately identify phishing sites with minimal error rates. The ANN also demonstrated excellent performance with an accuracy of 96.92% and an ROC-AUC of 0.9969, indicating near-perfect classification capacity. Meanwhile, MLP achieved the second-highest recall of 97.99%, but its precision was slightly lower than ANN and CNN. Autoencoder, despite having the lowest accuracy of 95.91%, maintained stable performance in detecting phishing. CNN ranked in the middle with an accuracy of 96.41% and an F1-score of 96.79%, demonstrating a good balance between precision and recall. Overall, all models recorded ROC-AUC values above 0.99, confirming the effectiveness of this SHAP-based framework in detecting phishing with a high level of interpretability. Comparison as show in Table 6.

Table 7. Comparison of Algorithm Performance Model

Model	Accuracy	Precision	Recall	F1-Score
ANN	0.969247	0.975785	0.968811	0.972286
MLP	0.968524	0.964194	0.979857	0.971963
Autoencoder	0.959117	0.957051	0.97011	0.963537
CNN	0.964182	0.965718	0.97011	0.967909
Bi-LSTM	0.970333	0.963128	0.984405	0.97365

The results show that the Bi-LSTM model has the best performance with an F1-score of 0.972 and an ROC-AUC of 0.993. This demonstrates Bi-LSTM's ability to capture temporal sequences and patterns between features, even when the data is tabular. CNN also demonstrates competitive performance, thanks to its ability to extract spatial representations between features.

3.2.2. Visualization of ROC Curve and Confusion Matrix

- ROC Curve Visualization:

Based on the ROC curve visualization in Fig. 6, the performance of the Bi-LSTM model in detecting phishing websites shows very optimal results compared to ANN, MLP, Autoencoder, and CNN. The ROC curve is almost entirely near the upper left corner of the graph, indicating a high true positive rate and a low false positive rate. An Area Under the Curve (AUC) value of 1.00 confirms that the model has perfect discrimination ability between phishing and non-phishing classes. The closer the AUC value is to 1, the better the model's performance in distinguishing between the two classes. This visualization strengthens the evidence that the developed Bi-LSTM framework is not only effective in prediction but also reliable in minimizing classification errors, making it very suitable for deep learning-based phishing detection applications that prioritize high accuracy and interpretability.

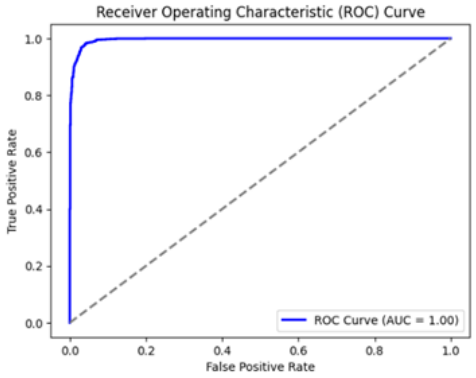


Fig. 6. Bi-LSTM ROC Curve Visualization

- Confussion Matrix

Based on the confusion matrix results for the Bi-LSTM model in Fig. 7, the model successfully classified 1167 non-phishing samples correctly (True Negative) and 1515 phishing samples properly (True Positive). There were 58 non-phishing samples that were incorrectly classified as phishing (false positive) and 24 phishing samples that were incorrectly classified as non-phishing (false negative). These results indicate that the model has excellent ability to distinguish between phishing and non-phishing sites, with a relatively low error rate for both types of misclassification. A smaller false negative ratio compared to false positive is very important in the context of phishing detection, because it reduces the risk of phishing sites going undetected.

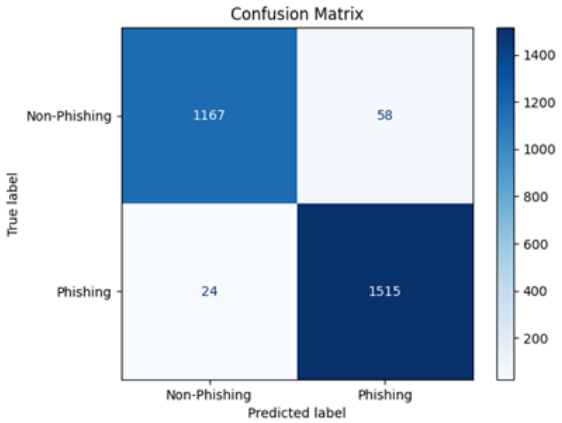


Fig. 7. Confusion Matrix Bi-LSTM

3.2.3. Model Interpretability with SHAP

The SHAP (SHapley Additive exPlanations) method is used to analyze how features contribute to predictions, ensuring that the model is both accurate and interpretable.

- SHAP Summary Plot

  Based on the SHAP summary plot in Fig. 8, the "URL_of_Anchor" feature has the most dominant influence on the Bi-LSTM model's output in detecting phishing sites. The red and blue dots indicate the distribution of feature values, with red representing high features and blue representing low ones. The distribution of these dots along the X-axis indicates the magnitude of the impact (either positive or negative) on the prediction.

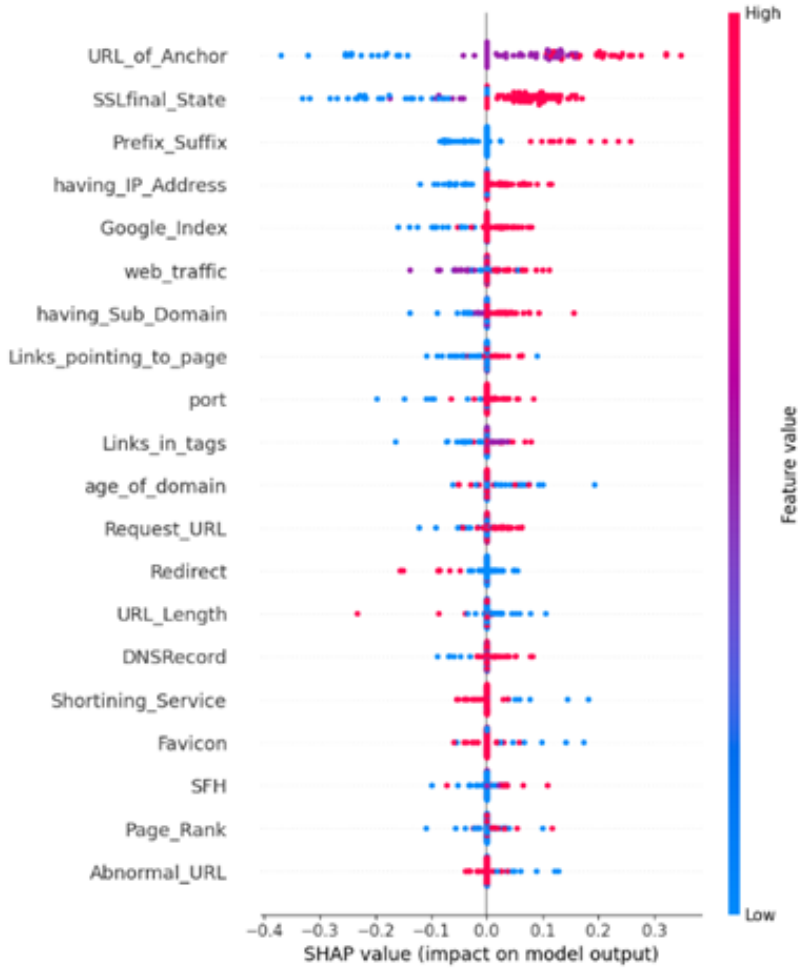


Fig. 8. SHAP Summary Plot

Furthermore, the "SSLfinal_State" and "Prefix_Suffix" features also contributed significantly, indicating that SSL validity and the presence of hyphens in the URL are important indicators in distinguishing phishing from legitimate sites. Other features such as "having_IP_Address," "Google_Index," and "web_traffic" also had a substantial influence on the model, indicating that characteristics related to the site's infrastructure and its presence in search engine indexes are also relevant for detection.

Features with large SHAP values (such as “URL_of_Anchor” and “SSLfinal_State”) show that high or low feature values can differentially direct the model's predictions toward phishing or non-

phishing classes. This demonstrates the flexibility of Bi-LSTM in capturing complex patterns from input features.

- SHAP Heatmap

The SHAP heatmap shows the absolute average influence of each feature on the model's predictions, allowing for numerical identification of important features.

Based on the SHAP heatmap in Fig. 9, the Bi-LSTM model shows that several features significantly contribute to the phishing site classification process. The "URL_of_Anchor" feature ranks highest in terms of absolute average SHAP value, indicating that the presence of anchor hyperlinks pointing to suspicious domains significantly impacts the model's decisions. Furthermore, the "SSLfinal_State" and "Request_URL" features also demonstrate high influence, reflecting that characteristics related to SSL certificate validity and resource calls from external domains are crucial in distinguishing phishing sites from legitimate ones.

Several other features, such as "having_Sub_Domain," "Abnormal_URL," and "Redirect," also demonstrate significant contributions, indicating that unusual URL patterns, the presence of excessive subdomains, and URL redirection mechanisms can be strong indicators in phishing detection. On the other hand, features such as "iframe," "RightClick," and "Statistical_report" have relatively small SHAP values, indicating that these features contribute less to the model's predictions.

Overall, these interpretability results confirm that the Bi-LSTM model is not only capable of producing accurate predictions but can also be logically explained by identifying important features that are semantically relevant to the characteristics of phishing sites. This strengthens the reliability of the proposed deep learning-based phishing detection framework and increases transparency in the model's decision-making.

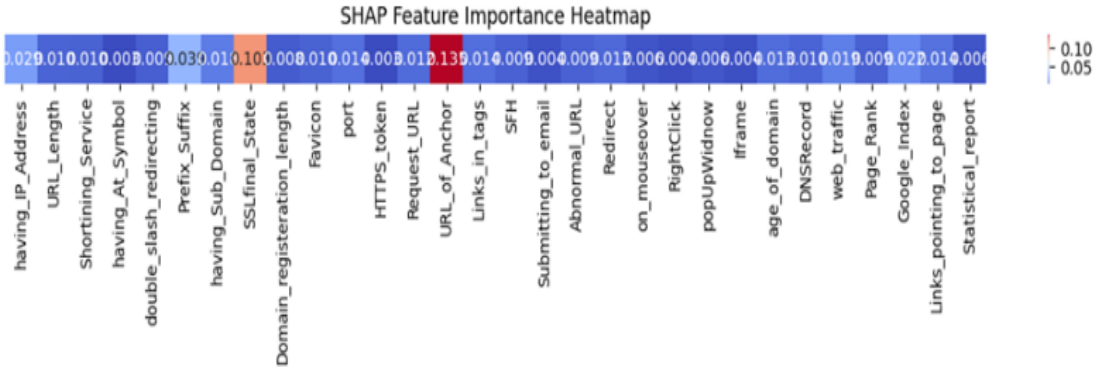


Fig. 9. SHAP Bi-LSTM heatmap

3.2.4. Comparative Interpretability Discussion

Beyond the Bi-LSTM model, SHAP analyses were also conducted on other deep-learning architectures to compare their interpretive behavior. Fig. 10 summarizes the distribution of global SHAP feature importances across all models. While the dominant features remain largely consistent, the degree and direction of influence vary by architecture.

The Bi-LSTM model produced the most stable and domain-aligned attributions, consistently assigning high positive SHAP values to URL_of_Anchor, SSLfinal_State, and Prefix_Suffix, which

correspond to well-known phishing indicators such as deceptive link embedding and invalid SSL certificates. Its bidirectional temporal encoding captures feature interdependencies effectively, explaining its superior predictive recall.

The CNN model also identified structural features such as Request_URL and Abnormal_URL as critical, but its focus tended toward localized feature interactions, likely due to convolutional kernel filters emphasizing specific attribute groups. This explains CNN's high precision but slightly lower recall.

In contrast, the Autoencoder model revealed dispersed SHAP attributions, indicating a broader but weaker feature sensitivity. This pattern suggests that while the latent representation generalizes well, it lacks fine-grained interpretability for phishing-specific cues. Similarly, the ANN and MLP models prioritized surface-level lexical attributes such as having_Sub_Domain, demonstrating simpler linear-like attribution patterns.

Collectively, these findings emphasize that although all models yield comparable predictive accuracy, their explanatory depth differs. Bi-LSTM offers richer, sequentially aware explanations, while CNN provides more spatially compact interpretations. Such comparative explainability is valuable for practitioners selecting between high-performance and interpretable architectures

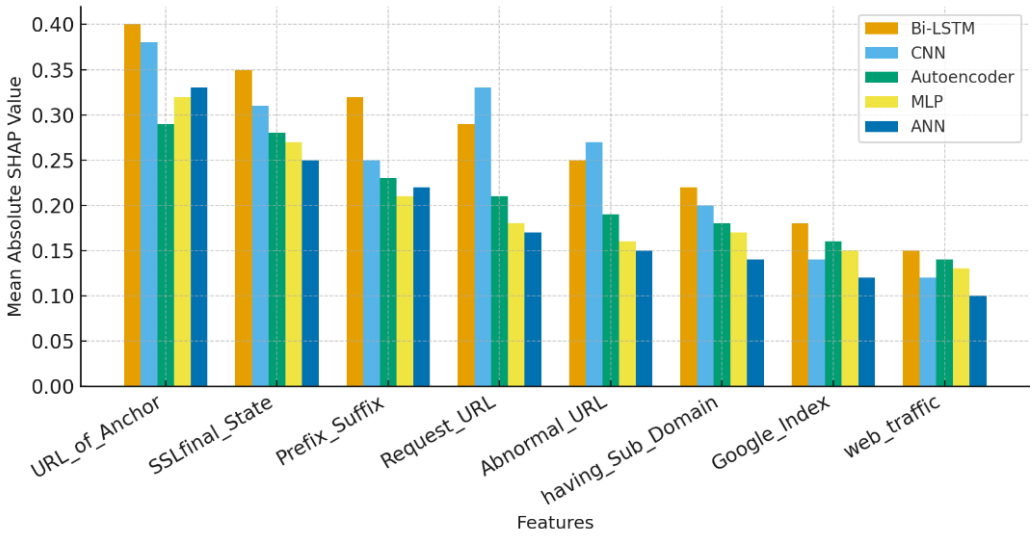


Fig. 10. Comparative SHAP feature importance distribution across deep-learning architectures

3.2.5. Local SHAP Instance Visualization (Force Plot)

To complement the global interpretation, Figure 11 illustrates a local SHAP force plot for a single test instance classified as phishing by the Bi-LSTM model. This visualization decomposes the model's prediction into additive feature contributions relative to the expected (base) output.

The force plot reveals that URL_of_Anchor and SSLfinal_State strongly push the prediction toward the phishing class, contributing +0.37 and +0.28 respectively to the output logit. In contrast, features such as Google_Index (–0.19) and web_traffic (–0.14) exert counteracting forces toward the legitimate class. The final model output probability (0.97) reflects the cumulative influence of these opposing forces.

This instance-level visualization demonstrates how SHAP can explain a model's decision in intuitive terms. Security analysts can trace which features cause the model to flag a website as phishing and validate

whether these reasons align with known attack behaviors. Thus, SHAP's local explanations transform the deep-learning output from a black box into a transparent, auditable decision process.

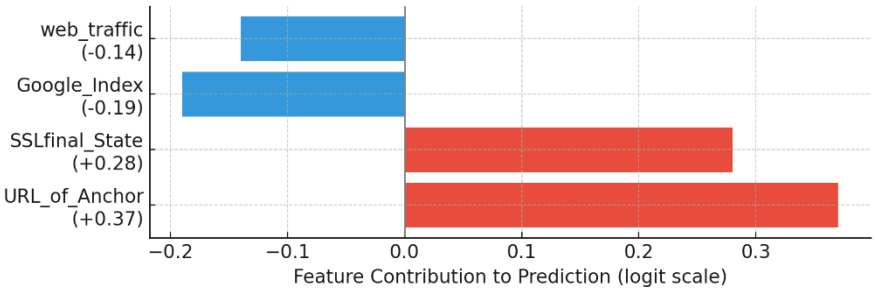


Fig. 11. SHAP local force plot for a sample phishing URL classified by Bi-LSTM

3.3. Research Comparison

Table 7 presents a comparison between previous research and the proposed approach, which utilizes the deep learning method and SHAP interpretation technique, along with an analysis of each model's performance in identifying phishing sites.

Table 8. compares previous studies to the proposed approach that utilizes SHAP interpretation techniques

Researcher	Method	Dataset	Model	Interpretabiliy	Accuracy	Advantages	Disadvantages
Alsufyani and Alajmani (2025) [47]	Deep Learning	PhishTank, 20.000 record	CNN, BiGRU, and GRU	Not available	0.953 (GRU)	CNN demonstrates strong capability in capturing sequential patterns within URLs commonly found in SMS-based phishing attacks	While CNN offers faster training time, it underperforms in terms of classification accuracy compared to GRU
Mia, (2024) [48]	Machine Learning + SHAP	Collected from Vrbančič et al. 88.647 record and Kaggle Dataset, 19.431 record	XGB	SHAP	0.971	Provides comprehensive SHAP-based analysis of feature importance and stability across heterogeneous phishing datasets.	Detection performance and feature relevance may be highly dependent on the dataset used, limiting model generalizability
Lakshmi et al, (2021) [49]	Deep Learning +	UCI Machine Learning	Deep Nerural Network	Not available	0.963	Utilizes a comprehensive feature	Lacks model interpretability, which limits

Researcher	Method	Dataset	Model	Interpretabiliy	Accuracy	Advantages	Disadvantages
	ADAM + Hyperparameter Tuning	Repository, 11.055 record				vector of 30 parameters, including hyperlink patterns in HTML, to enhance phishing detection performance.	transparency and trust in deep learning-based decisions
Islam et al, (2024) [50]	Deep Learning + SHAP	PhishTank, 38.000 record, and University of New Brunswick, 35.000 record	1D Convolutional Neural Network (CNN)	SHAP	0.998	Incorporates SHAP-based interpretability to enhance model transparency and explain predictions from the CNN architecture.	High computational cost due to deep learning architecture and SHAP post-hoc analysis, impacting training efficiency
Own Research (2025)	Deep Learning + SHAP	UCI Machine Learning Repository, 11,055 records	ANN, MLP, Autoencoder, CNN, Bi-LSTM	SHAP	0.970 (Bi-LSTM)	Combines multiple deep learning models with SHAP analysis to achieve high predictive performance and interpretable results, with Bi-LSTM yielding the highest accuracy	Training time is relatively high, particularly for sequential models like Bi-LSTM, due to their complexity and bidirectional structure

4. Conclusion

According to the findings of this study, a SHAP-based interpretative framework for phishing website detection was effectively constructed and evaluated by utilizing five different deep learning models. Compared to the other evaluated models, the Bi-LSTM model demonstrated superior classification performance based on the most commonly used evaluation measures. SHAP was also effective in showing how features contributed to predictions, which made the model's decisions clearer. This approach

addresses the necessity for detection systems that are not only accurate but also understandable. Such accuracy is an essential requirement for real-world implementations, notably in the fields of data regulation and cybersecurity. In the future, the framework that was developed has the potential to become the basis for intelligent defensive systems that are designed to protect against phishing threats.

Acknowledgment

The authors thank LPPM Universitas Amikom Purwokerto for supporting this research.

Declarations

Author contribution. All authors contributed equally to the main contributor to this paper. All authors read and approved the final paper.

Funding statement. None of the authors have received any funding or grants from any institution or funding body for the research.

Conflict of interest. The authors declare no conflict of interest.

Additional information. No additional information is available for this paper.

References

[1] "Phishing Reaches All-Time High in Early 2022," no. June, pp. 1–13, 2022, [Online]. Available at: https://docs.apwg.org/reports/apwg_trends_report_q1_2022.pdf.

[2] A. Safi and S. Singh, "A systematic literature review on phishing website detection techniques," *J. King Saud Univ. - Comput. Inf. Sci.*, vol. 35, no. 2, pp. 590–611, Feb. 2023, doi: 10.1016/j.jksuci.2023.01.004.

[3] G. Sonowal and K. S. Kuppusamy, "PhiDMA – A phishing detection model with multi-filter approach," *J. King Saud Univ. - Comput. Inf. Sci.*, vol. 32, no. 1, pp. 99–112, Jan. 2020, doi: 10.1016/j.jksuci.2017.07.005.

[4] R. Wahyudi, H. Marcos, U. Hasanah, B. P. Hartato, T. Astuti, and R. A. Prasetyo, "Algorithm Evaluation for Classification 'Phishing Website' Using Several Classification Algorithms," in *2018 3rd International Conference on Information Technology, Information System and Electrical Engineering (ICITISEE)*, Nov. 2018, pp. 265–270, doi: 10.1109/ICITISEE.2018.8720975.

[5] A. Basit, M. Zafar, X. Liu, A. R. Javed, Z. Jalil, and K. Kifayat, "A comprehensive survey of AI-enabled phishing attacks detection techniques," *Telecommun. Syst.*, vol. 76, no. 1, pp. 139–154, Jan. 2021, doi: 10.1007/s11235-020-00733-2.

[6] R. Verma and A. Das, "What's in a URL : Fast Feature Extraction and Malicious URL Detection," in *Proceedings of the 3rd ACM on International Workshop on Security And Privacy Analytics*, Mar. 2017, pp. 55–63, doi: 10.1145/3041008.3041016.

[7] N. Q. Do, A. Selamat, O. Krejcar, E. Herrera-Viedma, and H. Fujita, "Deep Learning for Phishing Detection: Taxonomy, Current Challenges and Future Directions," *IEEE Access*, vol. 10, pp. 36429–36463, 2022, doi: 10.1109/ACCESS.2022.3151903.

[8] A. Aljofey, Q. Jiang, Q. Qu, M. Huang, and J.-P. Niyigena, "An Effective Phishing Detection Model Based on Character Level Convolutional Neural Network from URL," *Electronics*, vol. 9, no. 9, p. 1514, Sep. 2020, doi: 10.3390/electronics9091514.

[9] Md Kamrul Hasan Chy, "Securing the web: Machine learning's role in predicting and preventing phishing attacks," *Int. J. Sci. Res. Arch.*, vol. 13, no. 1, pp. 1004–1011, Sep. 2024, doi: 10.30574/ijsra.2024.13.1.1770.

[10] S. Dargan, M. Kumar, M. R. Ayyagari, and G. Kumar, "A Survey of Deep Learning and Its Applications: A New Paradigm to Machine Learning," *Arch. Comput. Methods Eng.*, vol. 27, no. 4, pp. 1071–1092, Sep. 2020, doi: 10.1007/s11831-019-09344-w.

[11] S. F. Ahmed *et al.*, "Deep learning modelling techniques: current progress, applications, advantages, and challenges," *Artif. Intell. Rev.*, vol. 56, no. 11, pp. 13521–13617, Nov. 2023, doi: 10.1007/s10462-023-10466-8.

[12] A. C. Bahnsen, E. C. Bohorquez, S. Villegas, J. Vargas, and F. A. Gonzalez, "Classifying phishing URLs using recurrent neural networks," in *2017 APWG Symposium on Electronic Crime Research (eCrime)*, Apr. 2017, pp. 1–8, doi: 10.1109/ECRIME.2017.7945048.

[13] O. Al Harbi and A. Hamed, "A dual-stage deep learning model based on a sparse autoencoder and layered deep classifier for intrusion detection with imbalanced data," *Int. J. Sens. Networks*, vol. 45, no. 2, pp. 74–86, 2024, doi: 10.1504/IJSNET.2024.138918.

[14] S. Akkepalli and K. Sagar, "Anomaly-Based Network Intrusion Detection Using Hybrid CNN, Bi-LSTM Deep Learning Techniques," in *2024 4th International Conference on Innovative Research in Applied Science, Engineering and Technology (IRASET)*, May 2024, pp. 1–6, doi: 10.1109/IRASET60544.2024.10548678.

[15] T. T. Teoh, G. Chiew, E. J. Franco, P. C. Ng, M. . Benjamin, and Y. J. Goh, "Anomaly detection in cyber security attacks on networks using MLP deep learning," in *2018 International Conference on Smart Computing and Electronic Enterprise (ICSCEE)*, Jul. 2018, pp. 1–5, doi: 10.1109/ICSCEE.2018.8538395.

[16] I. H. Sarker, "Deep Cybersecurity: A Comprehensive Overview from Neural Network and Deep Learning Perspective," *SN Comput. Sci.*, vol. 2, no. 3, p. 154, May 2021, doi: 10.1007/s42979-021-00535-6.

[17] V. Buhrmester, D. Münch, and M. Arens, "Analysis of Explainers of Black Box Deep Neural Networks for Computer Vision: A Survey," *Mach. Learn. Knowl. Extr.*, vol. 3, no. 4, pp. 966–989, Dec. 2021, doi: 10.3390/make3040048.

[18] T. Qamar and N. Z. Bawany, "Understanding the black-box: towards interpretable and reliable deep learning models," *PeerJ Comput. Sci.*, vol. 9, p. e1629, Nov. 2023, doi: 10.7717/peerj-cs.1629.

[19] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, "A Survey of Methods for Explaining Black Box Models," *ACM Comput. Surv.*, vol. 51, no. 5, pp. 1–42, Sep. 2019, doi: 10.1145/3236009.

[20] Y. Nohara, K. Matsumoto, H. Soejima, and N. Nakashima, "Explanation of machine learning models using shapley additive explanation and application for real data in hospital," *Comput. Methods Programs Biomed.*, vol. 214, p. 106584, Feb. 2022, doi: 10.1016/j.cmpb.2021.106584.

[21] I. U. Ekanayake, D. P. P. Meddage, and U. Rathnayake, "A novel approach to explain the black-box nature of machine learning in compressive strength predictions of concrete using Shapley additive explanations (SHAP)," *Case Stud. Constr. Mater.*, vol. 16, p. e01059, Jun. 2022, doi: 10.1016/j.cscm.2022.e01059.

[22] C. Yang *et al.*, "How can SHAP (SHapley Additive exPlanations) interpretations improve deep learning based urban cellular automata model?," *Comput. Environ. Urban Syst.*, vol. 111, p. 102133, Jul. 2024, doi: 10.1016/j.compenvurbsys.2024.102133.

[23] R. Younisse, A. Ahmad, and Q. Abu Al-Haija, "Explaining Intrusion Detection-Based Convolutional Neural Networks Using Shapley Additive Explanations (SHAP)," *Big Data Cogn. Comput.*, vol. 6, no. 4, p. 126, Oct. 2022, doi: 10.3390/bdcc6040126.

[24] M. Al-Fayoumi, B. Alhijawi, Q. Abu Al-Haija, and R. Armoush, "XAI-PhD: Fortifying Trust of Phishing URL Detection Empowered by Shapley Additive Explanations," *Int. J. Online Biomed. Eng.*, vol. 20, no. 11, pp. 80–101, Aug. 2024, doi: 10.3991/ijoe.v20i11.49533.

[25] K. Roshan and A. Zafar, "Utilizing XAI Technique to Improve Autoencoder based Model for Computer Network Anomaly Detection with Shapley Additive Explanation(SHAP)," *Int. J. Comput. Networks Commun.*, vol. 13, no. 6, pp. 109–128, Sep. 2021, doi: 10.5121/ijcnc.2021.13607.

[26] T. Ahmad, P. Katari, A. K. P. Venkata, C. S. Ravi, and M. Shaik, "Explainable AI: Interpreting Deep Learning Models for Decision Support," *Adv. Deep Learn. Tech.*, vol. 4, no. 1, pp. 80–108, Feb. 2024. [Online]. Available at: https://thesciencebrigade.org/adlt/article/view/328.

[27] K. Saeed Alketbi and A. Mehmood, "A Comprehensive Survey of Explainable Artificial Intelligence Techniques for Malicious Insider Threat Detection," *IEEE Access*, vol. 13, pp. 121772–121798, 2025, doi: 10.1109/ACCESS.2025.3587114.

[28] "Phishing Websites," *UCI Machine Learning Repository*, 2015. [Online]. Available at: https://archive.ics.uci.edu/dataset/327/phishing+websites.

[29] K. Cabello-Solorzano, I. Ortigosa de Araujo, M. Peña, L. Correia, and A. J. Tallón-Ballesteros, "The Impact of Data Normalization on the Accuracy of Machine Learning Algorithms: A Comparative Analysis," in *Lecture Notes in Networks and Systems*, vol. 750 LNNS, Springer, Cham, 2023, pp. 344–353, doi: 10.1007/978-3-031-42536-3_33.

[30] A. Demircioğlu, "The effect of feature normalization methods in radiomics," *Insights Imaging*, vol. 15, no. 1, p. 2, Jan. 2024, doi: 10.1186/s13244-023-01575-7.

[31] A. Gicic, D. Đonko, and A. Subasi, "Time Sequence Deep Learning Model for Ubiquitous Tabular Data with Unique 3D Tensors Manipulation," *Entropy*, vol. 26, no. 9, p. 783, Sep. 2024, doi: 10.3390/e26090783.

[32] I. Nusrat and S.-B. Jang, "A Comparison of Regularization Techniques in Deep Neural Networks," *Symmetry (Basel).*, vol. 10, no. 11, p. 648, Nov. 2018, doi: 10.3390/sym10110648.

[33] C. Garbin, X. Zhu, and O. Marques, "Dropout vs. batch normalization: an empirical study of their impact to deep learning," *Multimed. Tools Appl.*, vol. 79, no. 19–20, pp. 12777–12815, May 2020, doi: 10.1007/s11042-019-08453-9.

[34] J. Naskath, G. Sivakamasundari, and A. A. S. Begum, "A Study on Different Deep Learning Algorithms Used in Deep Neural Nets: MLP SOM and DBN," *Wirel. Pers. Commun.*, vol. 128, no. 4, pp. 2913–2936, Feb. 2023, doi: 10.1007/s11277-022-10079-4.

[35] D. Majnarić, S. Baressi Šegota, I. Lorencin, and Z. Car, "Prediction of main particulars of container ships using artificial intelligence algorithms," *Ocean Eng.*, vol. 265, p. 112571, Dec. 2022, doi: 10.1016/j.oceaneng.2022.112571.

[36] P. Li, Y. Pei, and J. Li, "A comprehensive survey on design and application of autoencoder in deep learning," *Appl. Soft Comput.*, vol. 138, p. 110176, May 2023, doi: 10.1016/j.asoc.2023.110176.

[37] E. U. H. Qazi, A. Almorjan, and T. Zia, "A One-Dimensional Convolutional Neural Network (1D-CNN) Based Deep Learning System for Network Intrusion Detection," *Appl. Sci.*, vol. 12, no. 16, p. 7986, Aug. 2022, doi: 10.3390/app12167986.

[38] S. M. Hasanat *et al.*, "Enhancing Load Forecasting Accuracy in Smart Grids: A Novel Parallel Multichannel Network Approach Using 1D CNN and Bi-LSTM Models," *Int. J. Energy Res.*, vol. 2024, no. 1, p. 2403847, Jan. 2024, doi: 10.1155/2024/2403847.

[39] C. Ounoughi and S. Ben Yahia, "Sequence to sequence hybrid Bi-LSTM model for traffic speed prediction," *Expert Syst. Appl.*, vol. 236, p. 121325, Feb. 2024, doi: 10.1016/j.eswa.2023.121325.

[40] B. Sabiri, B. El Asri, and M. Rhanoui, "Mechanism of Overfitting Avoidance Techniques for Training Deep Neural Networks," in *Proceedings of the 24th International Conference on Enterprise Information Systems*, 2022, vol. 1, pp. 418–427, doi: 10.5220/0011114900003179.

[41] D. Sehrawat and Y. Singh, "Comparative Analysis on Fraud Detection in Credit Card Transaction Using Different Machine Learning Algorithms," in *Lecture Notes in Networks and Systems*, vol. 425, Springer, Singapore, 2022, pp. 673–684, doi: 10.1007/978-981-19-0707-4_61.

[42] D. Miyamoto, H. Hazeyama, and Y. Kadobayashi, "An Evaluation of Machine Learning-Based Methods for Detection of Phishing Sites," in *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 5506 LNCS, no. PART 1, Springer, Berlin, Heidelberg, 2009, pp. 539–546, doi: 10.1007/978-3-642-02490-0_66.

[43] M. Vijayalakshmi, S. Mercy Shalinie, M. H. Yang, and R. M. U., "Web phishing detection techniques: a survey on the state-of-the-art, taxonomy and future directions," *IET Networks*, vol. 9, no. 5, pp. 235–246, Sep. 2020, doi: 10.1049/iet-net.2020.0078.

[44] L. Tang and Q. H. Mahmoud, "A Survey of Machine Learning-Based Solutions for Phishing Website Detection," *Mach. Learn. Knowl. Extr.*, vol. 3, no. 3, pp. 672–694, Aug. 2021, doi: 10.3390/make3030034.

[45] S. M. Lundberg, P. G. Allen, and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," NeurIPS Proceedings, 2017. [Online]. Available at: https://dl.acm.org/doi/pdf/10.5555/3295222.3295230.

[46] S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees," Nat Mach Intell, vol. 2, no. 1, pp. 56–67, Jan. 2020, doi: 10.1038/s42256-019-0138-9

[47] S. Alsufyani and S. Alajmani, "A Deep Learning for Arabic SMS Phishing Based on URLs Detection," *Int. J. Adv. Comput. Sci. Appl.*, vol. 16, no. 1, p. 388, Jan. 2025, doi: 10.14569/IJACSA.2025.0160138.

[48] M. Mia, D. Derakhshan, and M. M. A. Pritom, "Can Features for Phishing URL Detection Be Trusted Across Diverse Datasets? A Case Study with Explainable AI," in *Proceedings of the 11th International Conference on Networking, Systems, and Security*, Dec. 2024, pp. 137–145, doi: 10.1145/3704522.3704532.

[49] L. Lakshmi, M. P. Reddy, C. Santhaiah, and U. J. Reddy, "Smart Phishing Detection in Web Pages using Supervised Deep Learning Classification and Optimization Technique ADAM," *Wirel. Pers. Commun.*, vol. 118, no. 4, pp. 3549–3564, Jun. 2021, doi: 10.1007/s11277-021-08196-7.

[50] M. R. Islam, M. M. Islam, M. S. Afrin, A. Antara, N. Tabassum, and A. Amin, "PhishGuard: A Convolutional Neural Network-Based Model for Detecting Phishing URLs with Explainability Analysis," in *2024 3rd International Conference on Artificial Intelligence For Internet of Things (AIIoT)*, May 2024, pp. 1–6, doi: 10.1109/AIIoT58432.2024.10574688